

NEXUS55 Gate5 配布資料 | 正常系だけの仕様書から例外系を補完する

要件定義 粒度 70% / 制限時間 2.0 時間

Table of Contents

0. この課題について

●課題の概要

項目	内容
Gate	Gate5
粒度	70%（正常系だけ書いてある）
テーマ	IT 資産管理システム／PC 貸出処理
制限時間	2.0 時間
判定方法	提出課題（コード＋モック画像）のみで判定。100 点満点中 80 点以上、かつ担当 AD 承認で合格

●この課題のねらい

- ・ 「この通り作ったら困る箇所」を、**8つの観点**を使って体系的に洗い出す
- ・ 異常系・境界値・同時実行という、画面を見ているだけでは絶対に出てこない観点を身につける
- ・ 「エラーにする」で止めず、**いつ検知して画面に何と出すか**まで決めて書く

●Gate1 との違い

	Gate1	Gate5
仕様	全部書いてある	正常系だけ
あなたの役割	忠実に再現する	足りない仕様を決める

評価軸	仕様の再現度	洗い出した観点の網羅性
禁止	勝手に足す・削る	「エラーにする」で止めること

●作業の流れ

#	STEP	内容	目安
1	配布資料を読む	正常系しか書かれていない仕様書を読む	10 分
2	例外系を洗い出す	8 観点シートを順に使い、最低 20 項目を洗い出す	30 分
3	例外系仕様書を作る	Gate1 の要件定義書と同じ形式にまとめる	25 分
4	実装する	優先度「高」（データが壊れるもの）から実装する	50 分
5	モックを取得し提出	エラー時の画面を 3 枚以上キャプチャして提出	15 分

●提出物

提出物	内容
ソースコード	優先度「高」の項目から実装。全項目の実装は求めない
モック画像	正常時に加えて、 エラー時の画面を 3 枚以上
例外系洗い出し表	8 観点で最低 20 項目。状況・起きる問題・決めた仕様・画面表示まで記入
例外系仕様書	バリデーション一覧／業務ルール／エラーメッセージ／状態遷移表の 4 点
補完後のフロー図	配布された正常系フロー図に、洗い出した分岐を書き足したもの（手書き可）
README.md	未実装項目とその理由／振り返り

●これを満たさないと不合格

- ☐ 洗い出しが8 観点のうち **5 カテゴリ以上**に及んでいること
- ☐ 二重貸出が防止されていること（同時実行・連打のいずれか）
- ☐ 複数テーブル更新が1 トランザクションになっていること
- ☐ エラー時のモック画像が **3 枚以上**提出されていること

●採点内訳（100 点）

観点	見るポイント	配点
例外系の網羅性	8 観点のうち何カテゴリをカバーできたか（ 項目数より偏りの無さ ）	30
優先度「高」の実装	二重貸出・トランザクションなどデータ破壊系を防げているか	25
エラー時の画面表示	いつ検知し、画面に何と出すかまで決めて実装されているか	20
モック画像	エラー時の画面が3 枚以上そろっているか	15
README・プロンプトログ	洗い出し表と実装の対応、未実装の理由が書かれているか	10

1. 状況設定

あなたはIT 資産管理システムの開発チームに配属されました。前任者が退職し、書きかけの仕様書だけが残されています。

仕様書には「ボタンを押したら貸出完了画面に遷移する」といった正常系しか書かれていません。在庫が0 のとき、すでに貸出中の端末を選んだとき、2 人が同時に操作したとき——何も書かれていません。

リーダーからの指示

「前の人が作った仕様書、正常系しか書いてないんだよね。異常系は自分で考えて埋めて。作る前に一回見せて。」

●対象システム

IT 資産管理システムの「PC 貸出処理」。社員が社内の共用ノート PC を一時的に借りる仕組みです。

2. PC 貸出機能 仕様書（正常系のみ・未完成）

これが配布される仕様書のすべてです。

2.1 概要

社員が共用ノート PC を貸出申請し、返却するまでを管理する。

2.2 データ定義

devices（端末）テーブル

カラム名	型	説明
id	BIGINT	主キー
asset_no	VARCHAR(20)	資産番号（例：PC-0001）
model_name	VARCHAR(50)	機種名
device_type	VARCHAR(20)	‘LAPTOP’ / ‘TABLET’ / ‘MONITOR’
status	VARCHAR(20)	‘AVAILABLE’ / ‘LENT’ / ‘REPAIR’ / ‘DISPOSED’
purchased_at	DATE	購入日

lendings（貸出）テーブル

カラム名	型	説明
------	---	----

id	BIGINT	主キー
device_id	BIGINT	端末 ID
user_id	BIGINT	借用者 ID
lent_at	DATETIME	貸出日時
due_date	DATE	返却予定日
returned_at	DATETIME	返却日時（未返却は NULL）
purpose	VARCHAR(100)	利用目的

employees（社員）テーブル

カラム名	型	説明
id	BIGINT	主キー
name	VARCHAR (50)	氏名
department	VARCHAR (50)	部署名
employment_status	VARCHAR (20)	‘ACTIVE’ / ‘LEAVE’（休職） / ‘RETIRED’（退職）

2.3 画面フロー（正常系）

[貸出申請画面]

↓ 機種を選び、返却予定日と利用目的を入力して「貸出する」を押す

[確認画面]

↓ 「確定」を押す

[貸出完了画面] … 「PC-0003 を貸し出しました。返却予定日は 2026/08/05 です。」

[返却画面]

↓ 借りている端末を選んで「返却する」を押す

[返却完了画面] … 「PC-0003 を返却しました。」

2.4 処理内容（正常系）

貸出処理

1. `lendings` に新規レコードを登録する（`lent_at` = 現在日時、`returned_at` = NULL）
2. `devices.status` を ‘LENT’ に更新する
3. 貸出完了画面を表示する

返却処理

1. `lendings.returned_at` に現在日時を設定する
2. `devices.status` を 'AVAILABLE' に更新する
3. 返却完了画面を表示する

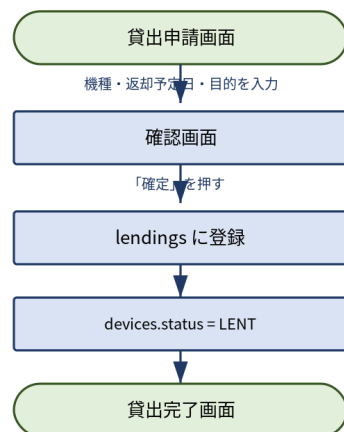
2.5 入力項目（正常系）

項目	形式
機種	プルダウン選択
返却予定日	日付入力
利用目的	テキスト入力

※ この仕様書はここまでしか書かれていません。

2.6 この仕様書を図にすると

課題2 | 渡された仕様書（正常系のみ）を図にすると



分岐（ひし形）が1つ也没有ありません。
＝ 想定外のことが何も起きない前提でしか描かれていない、ということです。

正常系みのフロー図

ひし形（判断）が1つ也没有ありません。これがこの課題の出発点です。

3. 例外系の洗い出し（8つの観点）

思いつきで考えると必ず偏ります。以下の8つの観点を順番に使ってください。

観点カテゴリ	問いかけ
① 在庫・状態	対象が存在しないときは？ すでに使われているときは？ 修理中・廃棄済みのときは？
② 人の状態	その人は本当にその操作をしてよい人か？ 退職・休職していたら？
③ 重複・多重	同じ人が2台借りようとしたら？ 同時に2人が同じ端末を選んだら？ ボタンを2回押したら？
④ 入力値	未入力なら？ 極端に長い値なら？ 過去の日付なら？ 10年後の日付なら？ 半角/全角/絵文字は？
⑤ 順序	借りていないものを返そうとしたら？ 返却済みのものをもう一度返したら？
⑥ 時間	返却予定日を過ぎたらどうなる？ 日付が変わる瞬間は？
⑦ 失敗時	2つのテーブル更新の途中で落ちたら？ 通信が切れたら？
⑧ 不正操作	ブラウザバックで確認画面に戻ってもう一度確定したら？ URLを直接叩かれたら？

●洗い出しシート（提出物）

最低 20 項目を目標にしてください。

No	観点	想定される状況	このままだと起きる問題	決めた仕様	画面表示	優先度
1	①	選択した端末が既に貸出中	二重貸出になり、追跡不能になる	貸出直前に status を再確認し、AVAILABLE でなければエラー	この端末は現在貸出中です。別の端末を選択してください。	高

「エラーにする」で止めないでください。「いつ検知して」「画面に何と出するか」まで書いて初めて仕様です。

●観点別チェックリスト

□ ① 在庫・状態

- ☐ 対象が存在しない（削除済み・不正な ID）
- ☐ 対象が既に使用中・占有されている
- ☐ 対象が利用不可の状態（修理中・廃棄済み）
- ☐ 対象が 0 件（プルダウンが空になる）
- ☐ 貸出対象外の device_type が選べてしまう

□ ② 人の状態

- ☐ 退職・休職している
- ☐ 上限台数に達している
- ☐ 他人の貸出を操作しようとしている
- ☐ 社員レコードが存在しない

□ ③ 重複・多重

- ☐ 同じ操作を 2 回行う
- ☐ ボタンの連打・二重送信
- ☐ 2 人が同時に同じ端末を選ぶ（競合状態）
- ☐ ブラウザバックからの再送信

□ ④ 入力値

- ☐ 未入力・空文字・空白のみ
- ☐ 桁あふれ（DB 定義を超える長さ）
- ☐ 存在しない日付（2026/02/30 など）
- ☐ 過去日・極端な未来日
- ☐ 境界値（当日、上限日ちょうど）
- ☐ 特殊文字（絵文字、機種依存文字、HTML タグ）

- ☐ プルダウンに無い値を送信

☐ ⑤ 順序

- ☐ 借りていない端末を返却しようとする
- ☐ 返却済みをもう一度返却する
- ☐ 貸出中の端末のステータスを変更する

☐ ⑥ 時間

- ☐ 延滞している
- ☐ 延滞中のユーザーが新規貸出する
- ☐ 締め時刻・日またぎの扱い

☐ ⑦ 失敗時

- ☐ 複数テーブル更新の途中で失敗（トランザクション）
- ☐ DB 接続断・タイムアウト
- ☐ 完了画面表示前にセッションが切れた

☐ ⑧ 不正操作

- ☐ URL 直打ちで確認画面をスキップ
 - ☐ 他人の ID を指定して実行
 - ☐ 確認画面と登録内容のすり替え
-

4. 例外系仕様書の作り方

洗い出した内容を、**Gate1** で配布された要件定義書と同じ形式にまとめてください。

●作成する 4 点 + 1 点

#	成果物	内容
1	バリデーション一覧表	項目・必須有無・型・桁数・範囲・エラーメッセージ
2	業務ルール一覧表	R-01 形式で採番

- | | | |
|---|-------------|----------------------------|
| 3 | エラーメッセージ一覧表 | 状況・文言・表示種別 |
| 4 | 状態遷移表 | devices.status の遷移可否マトリクス |
| 5 | 補完後のフロー図 | 配布された正常系フロー図に、洗い出した分岐を書き足す |

●バリデーション定義シートのひな形

画面	項目名	必須	型	最小	最大	使用可		業務	エラー
						文字	トリム	チェック	メッセージ
貸出申請	返却予定日	○	日付	翌日	90 日後	-	-	休業日は不可	返却予定日は明日から 90 日以内で入力してください。

貸出申
請

●フロー図について

5 番のフロー図が、この課題の成果を最も分かりやすく示します。ひし形がいくつ増えたかを数えて、README に書いてください。

手書きをスマートフォンで撮影して提出して構いません。

5. 実装の優先度

時間内に全部できなくて構いません。優先度の高いものから実装してください。

優先度	基準	例
高	データが壊れる／不整合が残る	二重貸出、トランザクション不整合
中	業務上おかしい	退職者への貸出、過去日付の返却予定日
低	見た目・使い勝手	エラーメッセージの文言調整、入力補助

優先度をつけるところまでが要件定義です。全部やろうとするのは、優先順位をつけていないのと同じです。

6. モック画像の提出要領

●必ず提出する画像（4枚）

#	画像	撮影する状態
1	正常時の貸出完了画面	貸出が成功し、完了メッセージが出ている状態
2	エラー① 状態チェック	既に貸出中の端末を選んで、エラーが出ている状態
3	エラー② 入力値チェック	返却予定日を過去日にして、エラーが出ている状態
4	エラー③ 任意	自分が実装した例外系のうち、最も自信のあるもの

●余裕があれば追加で提出する画像

#	画像	撮影する状態
5	貸出可能端末が0件	プルダウンが空にならず、メッセージが出ている状態
6	二重送信の防止	連打しても1件しか登録されないことが分かる状態（一覧画面など）
7	退職者の貸出	退職済み社員での貸出がブロックされている状態

●撮影のルール

- ファイル名は mock_02_エラー_貸出中.png のように、**何の状態かが分かる名前**にすること
 - エラーメッセージの文字が読める大きさにキャプチャすること
 - コンソールアプリの場合は、実行結果のターミナル画面をキャプチャすること
-

7. README のテンプレート

Gate5 提出物

1. 起動手順

(環境構築コマンドと起動コマンドを書く)

2. 洗い出しの結果

- 洗い出した項目数: ___ 件
- カバーした観点: ①②③④⑤⑥⑦⑧のうち ___ カテゴリ
- 補完後のフロー図で増えたひし形の数: ___ 個

3. 実装した項目 / 実装しなかった項目

No	項目	優先度	実装	理由
---	---	---	---	---

4. モック画像一覧

(ファイル名と、それが何の状態かの説明)

5. 振り返り

5-1. 自分の洗い出しで、項目が0件だった観点カテゴリはどれですか。

5-2. 「言われれば当たり前」だったのに気づけなかった項目を3つ挙げ、なぜ自力で気づけなかったかを書いてください。

5-3. Gate1の振り返りで書いた「確認に行くこと」リストと比べて、増えた観点は何か。

8. 注意事項

#	内容
---	----

- 1 「エラーにする」だけでは仕様になっていない。**表示文言まで決めること**
- 2 入力チェックばかり大量に出すのは典型的な失敗。**8 観点の偏りを無くす**
- 3 洗い出したのに実装で無視しないこと。実装しないなら理由を書く
- 4 全項目を実装しようとして1つも完成しないのが最悪のパターン
- 5 AI ツールの使用は自由。ただしプロンプトログを必ず残す
- 6 動かないコードでも提出する。未提出が一番評価できない